

Senior Python Engineer Assignment – Broker Network Gateway & Static IP Orchestrator (Updated)

Objective: Design and implement a production-grade Broker Network Gateway capable of managing multiple brokers, Vultr-hosted static IP orchestration, smart order routing, failover, monitoring and security.

Part 1 – Dynamic Broker Routing Engine

Maintain multiple broker connections and static IPs. Route orders using weighted routing based on latency, success rate, broker priority, IP health and automatic failover.

Part 2 – Static IP Orchestrator

Allocate, release, attach, detach and rotate static IPs behind an abstraction layer.

Part 3 – Broker Whitelist Synchronization

Normalize broker whitelist formats (JSON/XML), detect missing, duplicate, stale and unauthorized IPs.

Part 4 – Zero-Downtime IP Rotation

Rotate IPs without interrupting in-flight orders.

Part 5 – Broker Health Scoring

Compute broker health using latency, connectivity, timeout and success metrics.

Part 6 – Concurrent Order Engine

Support ~5000 orders/min using asyncio, queues, retries, exponential backoff and idempotency.

Part 7 – Distributed Locking

Prevent concurrent IP rotation using Redis-based distributed locking.

Part 8 – Vultr Infrastructure & IP Lifecycle Management

Provision/terminate Vultr instances, manage public IPs, maintain Client→Broker→Instance→IP mapping, stop auto-renewal on expiry, suspend/destroy infrastructure and prevent trading after expiry.

Part 9 – Failure Simulation

Simulate broker timeout, DNS failures, TLS failures, IP blacklisting and recovery.

Part 10 – Observability

Expose Prometheus metrics and structured JSON logs.

Part 11 – Event Driven Architecture

Publish events such as IPAllocated, BrokerDown and OrderSucceeded.

Part 12 – Security

JWT, RBAC, API Keys, encrypted configuration, HMAC signing and secret rotation.

Part 13 – Architecture Documentation

Architecture, HA/DR, monitoring, scaling and security documentation.

Part 14 – Multi-Broker Management

Onboard unlimited brokers without code changes using adapters.

Part 15 – Broker Capability Discovery

Use metadata for routing decisions.

Part 16 – Smart Order Routing

Route dynamically using health, latency, limits and capabilities.

Part 17 – Session Manager

Centralized token refresh with encrypted credential storage.

Part 18 – Rate Limit Manager

Track quotas, queue requests and reroute when limits are exceeded.

Part 19 – Broker Failover

Automatic broker switching with exactly-once submission.

Part 20 – Multi-Region Static IP Pools

Support multiple regions while respecting broker/IP constraints.

Part 21 – Dynamic Configuration

Apply broker configuration changes without restart.

Part 22 – Order Replay & Recovery

Recover pending orders after restart without duplicates.

Part 23 – Monitoring Dashboard APIs

Expose APIs for broker health, sessions, IP health, failovers and rate limits.

Part 24 – Design Challenge

Explain support for 50+ brokers, 100k+ orders/min, resilience and scalability.

Additional Requirements

Buy/Sell Pipeline APIs: Place Buy, Place Sell, Cancel Order, Order Status and Order History APIs. **Multi-Broker IP Management:** A client can own any number of brokers. **IP Allocation Rules:** A single static Vultr IP may be shared across multiple brokers. Once an IP has been

assigned/used for a specific broker, it cannot be reassigned to that same broker until released or rotated according to the configured reuse policy. Maintain Client → Broker → Assigned IP → Order mapping. Prevent duplicate/conflicting IP assignments. Record every IP allocation, release, rotation and order execution in an audit log. On subscription expiry, disable trading, stop Vultr auto-renewal, suspend/destroy infrastructure and block new orders. Every order must be routed through the broker's currently assigned IP.